

Component Design Document

X.25 Module

Subsystem: X.25 Module

Component Name: X.25 -TCP/IP Gateway

Author:	Kevin Guerra
Document Type:	CDD
Version:	1
Revision:	1.0a
Date:	Tuesday, June 20, 2000
Release:	1
DocLib Number:	1
File Name:	CD01xxxx
EPR/Feature List:	
Reason for Change:	

(template rev: 4, 1/11/99, DocLib # 975)

Change History:

Author	Revision	Date	Description of change(s). Include chapter #s

Table Of Contents

Introduction

1.1 Purpose Of Document

This document will list module structure, operation, features, advantages and disadvantages of the module and other relevant data.

1.2 Intended Audience

- Project managers
- Programmers
- System testers
- Anyone interested in X.25 connectivity solutions

1.3 Scope

This document provides detailed information on the module. It does not cover however details on DLLs that the module uses. For that information refer to Advanced Relay Corp. documentation.

- Function
 - Design
 - Line States
 - Messages
-

Component Design Document

X.25 Module

1.4 Terminology

1.5 Document Structure

Standard.

1.6 Related Documents

- LayGO Multiprotocol Toolkit Documentation, Advance Relay Corporation.
- WinSock2 Specification
- MFC documentation
- CNN Trace

2. Overview

2.1 General

X.25 Module is actually an application that acts as a gateway between TCP/IP and the X.25 Protocols. X.25 is a widely used protocol in legacy systems. The telecommunications industry relies on this protocol. However many companies are moving from X.25 to TCP/IP, many speculate this protocol would soon become obsolete. Nevertheless, because a great number of systems use this protocol, we must support it. X.25 Module will address two main issues: a) We must support X.25 and b) We must also move on to newer technologies. Therefore, the purpose of this module is to provide X.25 connectivity to any client in such a way that the client application 'thinks' it's a TCP/IP connection. Thus we can support X.25 while allowing other programmers to design their applications for TCP/IP connectivity. This method offers other advantages, the most obvious one being easy migration to TCP/IP.

The X.25 Module is an MFC Dialog Box Application. It has a GUI with menus and other controls which provide limited functionality (functionality is sort of automatic and services are provided to TCP/IP clients.)

The module uses Emulex xp8400 hardware which provides X.25 support. The drivers are provided by ARC (Advanced Relay Corp.) The network connectivity to X.25 (including physical, link, packet and PVC layers) is provided in an API in DLL's also by ARC.

Configuration information for the stack, system buffers, physical, link and packet layers is stored in several configuration files (*.cfg) in a location pointed to by the configuration database (MS Access database.) This configuration files contain parameters specific to X.25 such as DTE or DCE operation, number of PVCs, packet sizes, etc..., for complete information refer to ARC's documentation.

Each X.25 Line has a number of PVCs. For each PVC there is a TCP/IP listen socket, which remains listening until a connection is made for efficient resource utilization.

Application settings and configuration information is stored in an MS Access database. Access to this database is through ADO via a ODBC DSN. Connectivity to the database is stored in the registry. The database stores:

1. Defaults
 2. Connection types
-

Component Design Document

X.25 Module

3. Protocol names, and
4. Services. Each entry is an X.25 line and describes which *.cfg files to apply to it, how many PVCs it will use and its connection type [listen, connect, disabled]

The user interface provides icons and simulated LED's that reflect the current TCP/IP socket, X.25 Line and PVC states.

An admin listen socket is provided so that applications can control the module and request information for specific lines.

Care must be taken when configuring the lines. It has been known to ARC that the hardware has become damaged from incorrect settings. Also the number of PVCs must match per line.

Trace messages are generated for just about anything, except in places where this would cause the module to slow down. For information on Trace, refer to its document.

2.2 Context

Three possible contexts:

- User
- TCP/IP Client
- TCP/IP Client Admin

2.3 Interfaces

USER INTERFACE:

Menus:

- Startup. Initialize lines
- Shutdown. De-initialize lines
- Configuration Database. Call ADO dialog box to set connectivity parameters
- Exit.
- Statistics. Call dialog box to show statistics for current PVC.
- Debug. Show Trace dialog box.
- Debug Levels. Show info on levels.

The commands Startup and Shutdown allow the Admin client application to change parameters in the configuration database and apply those changes without requiring a module restart. However when the Shutdown command is executed all connections (including TCP/IP) are broken.

Spin button control. Cycles through all the available PVCs.

Command button. Executes command shown in right most list box for item in left most list box.

Icons. Reflect the current state for the line items.

LEDs. Reflect the current state for LED items as you would find them in a modem.

Text. The text below describes the current state for each line item.

ADMIN INTERFACE:

An application must connect to the TCP/IP port specified in the module application defaults database table. Once the connection is established the module can receive commands and send data to the Admin Client App.

Command allowed:

1. Get Line State [Xon, Xoff and Down]
 2. Set Line State, [Xon and Xoff]
 3. Get Line Statistics [for all layers simultaneously, the packet is more than 1k]
 4. Startup
-

Component Design Document

X.25 Module

5. Shutdown

Message In Format:

Bytes	Description
2	Line Number
1	Message Type
1	Message Code

Message Out Format:

Bytes	Description
2	Line Number
1	Message Type
1	Message Code
Variable	Message Text

CLIENT INTERFACE:

Applications that wish to use an X.25 PVC must connect to the module via TCP/IP. The listen socket ports are numbered after the Base Port parameter in the Defaults table in the configuration database. For example if you use base port 2000 that port will be the Admin port, and PVC #1 will be available on port 2001, PVC #2 on port 2002, etc....,

To send data to the PVC simply send the data on the connected port. Because currently the ARC API and the xp8400 driver do not support call back events, so we do not know when the data arrives, the module must poll for data events as fast as needed. That isn't necessarily a problem, though. TCP can process data with higher latency than X.25 and the new PC's CPUs are a magnitude of times faster than the data transmitted in either protocol anyway. So, when the module polls and finds data, sends it immediately through the client socket. This would give the client application the appearance that is dealing with a TCP/IP connection, indeed it is.

The TCP/IP connectivity is provided through a DLL that encapsulates CAsyncSocket and provides additional functionality:

- Network error reporting
- Buffering of data
- State transitions and notifications of them

If the socket is trying to send data faster than X.25 can transmit, hopefully the other end will notify us. If so, an Xoff request is made by the other end. A message is then sent to the client indicating a blocking condition. The client should handle this, but can choose to ignore. The data is then buffered up to a point (default table defined). When the buffer is about to fill up and data keeps arriving warnings are sent to the client. If the client ignores those warnings and we still have not processed the data, the data is then discarded and an error message is issued back to the client.

The protocol messages between the client and the module are conducted through the same TCP/IP connection, however it used the Out of Band Data mechanism. Clients connecting to the module should handle OOB data. They could ignore the data, at the risk of having the module discard data should the need arise. And they could not issue commands or request info from the module, of course.

If the client socket disconnects or the connection is lost the PVC will again enable the listen socket.

The message format is the same as that for the Admin client. With the following two differences:

Component Design Document

X.25 Module

1. Line Number is not used. The two bytes are NOT included [they are NOT zero]
2. Commands: Startup and Shutdown are not supported

The module was coded so that the X.25 PVC would be resilient and provide fail-safe operation. Thus you can disconnect the cables from the xp8400 card and reconnect them, and the module will attempt to re-establish the connections. This depends on other factors, however. If the line was configured to listen, then, the other end is responsible for reconnecting. This is not the same listen as the listening socket, though.

CONSIDERATIONS:

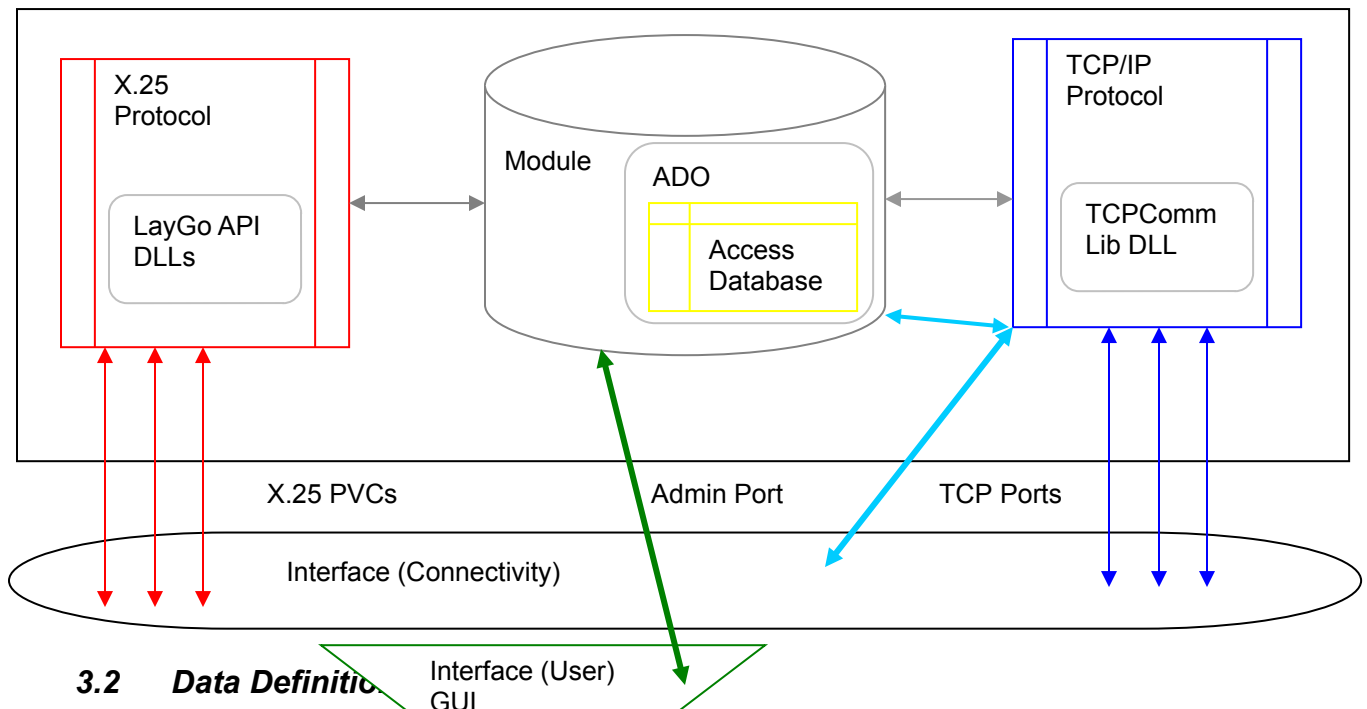
- Do not send TCP/IP packets that are bigger than the X.25 max packet size setting in the *.cfg file for that line. Doing so would cause an error.
- Lines that will not be used should be marked as disabled. The system uses resources and memory buffers for each active line, affecting performance.

TODO:

- Packet acknowledgement OOB message
- Polling Speed Setting in Defaults table.
- Blocking

3. Process Breakdown

Logical Data Flow



3.2 Data Definition

Database Structure:

Table: tblConnectionTypes

Name	Type	Size

Component Design Document

X.25 Module

iConnectionType	Number (Long)	4
szConnectionType	Text	50

Table: tblDefaults

Name	Type	Size
szConfigurationFile	Text	250
szConfigurationPath	Text	250
iBasePort	Number (Long)	4
bAutoStartup	Yes/No	1
iPollingRate	Number (Long)	4
iControlsRefreshRate	Number (Long)	4
bRefreshWhenInactive	Yes/No	1

Table: tblProtocols

Name	Type	Size
iProtocolID	Number (Long)	4
szProtocolName	Text	50

Table: tblServices

Name	Type	Size
iMajorDevice	Number (Long)	4
iProtocolPhysical	Number (Long)	4
szConfigurationFilePhysical	Text	250
iProtocolLink	Number (Long)	4
szConfigurationFileLink	Text	250
iProtocolPacket	Number (Long)	4
szConfigurationFilePacket	Text	250
iProtocoPVC	Number (Long)	4
iPVCCount	Number (Long)	4
iConnectionType	Number (Long)	4

3.3 Components

1. LayGo XP8400 X.25 Board NT Driver
2. LayGo API DLLs
3. LayGo configuration files
4. Configuration database, MS Access
5. TCP/IP communications DLL
6. X.25 module, EXE file
7. TCP/IP Client test utility

Feature Dependency

- The LayGo API DLLs access the X.25 Board driver services and the configuration files
 - The configuration database is accessed through ADO. ADO is accessed through a COM server and some DLLs
 - The module uses the LayGo and TCP/IP DLLs
 - You can use the test client utility to send and receive messages
-

Component Design Document

X.25 Module

4. Routine Breakdown

4.1 Introduction

The TCP/IP DLL encapsulates all protocol connectivity and provides automatic buffering. It uses CAsyncSocket.

The LayGo DLLs encapsulate X.25 setup, initialization, buffers creation and maintenance, state transitions, configuration and all X.25 connectivity issues.

The X.25 module is an MFC dialog box application. It has a timer that polls for data and events. It tries to ensure that we have a connection at all times and buffers data when the peer or the module are blocking and sends protocol messages as well as commands.

4.2 Module Hierarchy

See graphic above.

4.3 Module Descriptions

LayGo API, for a description and details refer to ARC's documentation

The TCP/IP DLL is an FSM (finite state machine). It is used as an CAsyncSocket but provides automatic buffering and notification of state changes

The X.25 module is structured as follows:

- CX25RelayApp, (CWinApp) Initializes and deinitializes COM and WinSock2
- CX25RelayDlg, (CDialog) provides GUI and contains X.25 manager object and CCNTrace object. Contains the timer function
- CX25Manager, starts up and shuts down, accesses configuration database and configuration files, sets up connectivity, contains X.25 line and PVC objects, calls event and data handling functions in each line and PVC object, contains admin socket which accepts commands
- CX25Line, is an FSM, contains network, link and packet layers and buffer used by its children PVCs, tries to ensure we have a good medium through which its PVCs can send and receive data, so it handles connections and network events
- CX25PVC, derived from CTCP Socket is an FSM, contains minor device (PVC), contains buffers used when peer or local are blocking, sends and receives and acts on protocol messages, exchanges data, basically most of the action happens here

Appendix
